Quantum-Assisted Disaster Relief Routing

Grover-based minimum search over multi-convoy relief plans on real road networks

Qiskit Quantum Optimization Challenge · Domain 2: Disaster Response · Repository: https://github.com/MARafey/Quantum-Routing

Generated 2026-11-12 from the committed result files

1. Summary

- **Problem.** Relief convoys leave a few logistics hubs and must reach many stricken towns. Which hub serves which town, and in what order, decides how long the last town waits for aid. With 8 to 10 towns and 1 to 3 hubs there are 1,814,400 to 3,628,800 possible plans.
- **Quantum approach.** Every plan gets an index. The index lives in a register of $\lceil \log_2 N \rceil$ = 21–22 qubits, and Dürr–Høyer minimum finding (Grover search with the BBHT schedule), implemented with Qiskit, looks for the plan with the smallest makespan, optionally warm-started by a classical heuristic. Travel times come from real OpenStreetMap roads through OSRM, in four regions covering six countries.
- **Result against exhaustive search.** In all four region runs the quantum search holds the optimal plan after examining 0.19–0.23% of the plans, about 436–537× fewer than brute force or its 8-process parallel version, which both examine 100%. Across a fit to 13 measured instances, queries grow as $3.51 \cdot \sqrt{N}$.
- **Honest limits.** The quantum stage is simulated on a laptop, so its wall-clock time is not a hardware result. A stronger classical method, exact subset dynamic programming, solves every one of these instances in 20–41 ms and needs fewer steps than plain Grover search beyond about 17 sites. We therefore claim a quadratic reduction in oracle queries over unstructured search, not a practical advantage.
- **Qubits.** The minimal register $\lceil \log_2 N \rceil$ is optimal: every extra qubit pads the search space with unmarked states and multiplies the queries by about 1.4 (Section 7.5).

2. Problem statement and motivation

Motivation. In the 2022 monsoon floods roughly a third of Pakistan was under water and more than 30 million people were affected. After the February 2023 earthquake, relief had to reach towns on both sides of the Türkiye–Syria border. In both disasters the same planning question arises: relief stocks sit in a handful of hubs, the stricken towns are spread over hundreds of kilometres, and every convoy-hour counts for the town served last.

Intended users. National and regional disaster-management authorities and relief logistics coordinators, who must decide convoy dispatch plans quickly and re-plan as damage reports arrive.

Optimisation problem. Choose, for every stricken town (site), which convoy (hub) serves it and in what order, so that the time until the last site is reached is minimal.

- **Measurable objective.** Makespan: the time at which the last site is reached, i.e. the maximum over convoys of route travel time. A tiny tie-break (0.001 × total travel time) prefers shorter total driving.
- **Constraints.** Every site is visited exactly once, by exactly one convoy; each convoy starts at its own hub; 1–3 hubs and 2–10 sites in the demonstrator.

- **Competing objectives.** Serving far towns first lowers their wait but delays the near ones; adding hubs shortens routes but splits the plan space further. Both appear in the instances below.

3. Formulation and test instances

Decision variables. A plan assigns each of n sites to one of k convoys and fixes the visiting order inside each convoy. Writing the sites in one sequence and cutting it into k consecutive parts gives every plan exactly once, so the number of plans is

$$N = n! \cdot C(n+k-1,\ k-1)$$

Each plan is encoded as one integer index in [0, N): the rank of the site permutation times the number of compositions plus the rank of the split. The mapping is a bijection (unit-tested by round trip), so one index is one plan.

Objective. minimise max over convoys v of T_v(plan) + 0.001 · Σ_v T_v(plan), where T_v is the travel time of convoy v along its route from its hub, using the OSRM driving-time matrix between all points.

Region (seeded random scenario)	Hubs × sites	Plans N	Qubits $\lceil \log_2 N \rceil$	Last site reached (h)
Pakistan floods	2 × 9	3,628,800	22	18.1
Türkiye–Syria earthquake	1 × 10	3,628,800	22	18.5
Bengal floods (Bangladesh + India)	3 × 8	1,814,400	21	8.1
Nepal–India Himalaya earthquake	3 × 8	1,814,400	21	6.3

Table 1. Test instances. Each is a reproducible random scenario (fixed seed) drawn from a pool of real cities; sites are spread over at least two provinces or countries.

4. Architecture

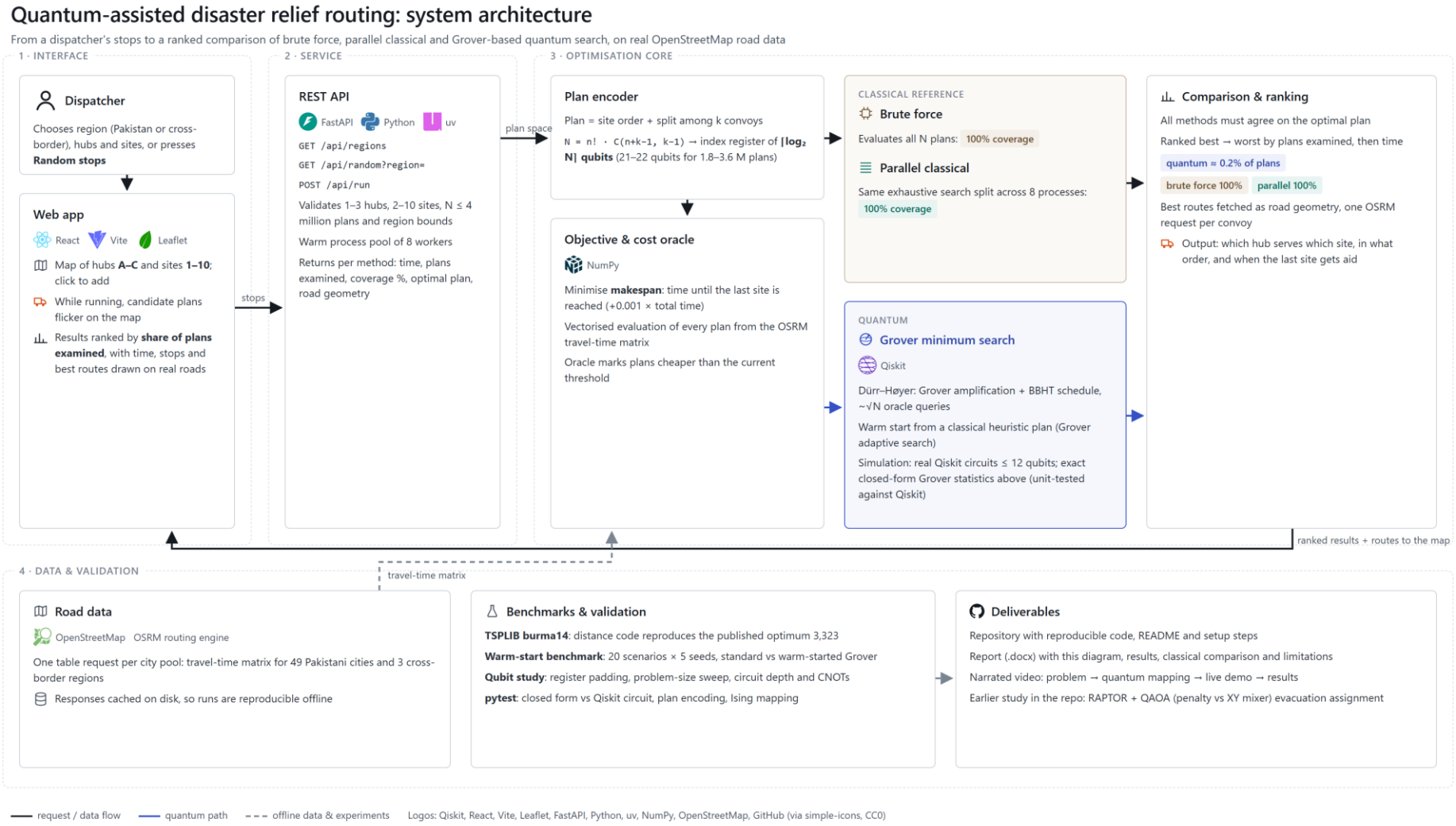


Figure 1. System architecture: interface, REST service, optimisation core (classical references and the quantum search), and data and validation. Logos: Qiskit, React, Vite, Leaflet, FastAPI, Python, uv, NumPy, OpenStreetMap, GitHub (simple-icons, CC0).

The dispatcher chooses a region and the hubs and sites (or presses Random stops). The web app sends them to a FastAPI service, which validates the request and slices the cached OSRM travel-time matrix. The optimisation core evaluates every plan classically (brute force, then the same search split over 8 processes, then exact subset DP as a stronger reference) and runs the quantum search over the plan index. The ranking compares how much of the plan space each method examined, and the best routes are fetched as real road geometry and drawn on the map.

5. Quantum approach and justification

5.1 Representation

The N plans are indexed by a register of $q = \lceil \log_2 N \rceil$ qubits (21 for 1.8 million plans, 22 for 3.6 million). Basis state $|i\rangle$ stands for plan i; indices ≥ N are never marked. The cost of plan i is a classical function C(i), so the objective can be any function and needs no quadratic form.

5.2 Algorithm: Dürr–Høyer minimum finding

- Pick a threshold plan y (random, or the warm-start plan).
- Mark all plans with C(i) < C(y) in an oracle, run j Grover iterations with j drawn by the BBHT schedule (growth factor 6/5, since the number of marked plans M is unknown), and measure.
- If the measured plan beats y, make it the new threshold and restart the schedule; otherwise grow the schedule.
- Stop after the budget $22.5 \cdot \sqrt{N} + 1.4 \cdot \log_2^2 N$ queries. Success means the optimum was held by then.

5.3 Why this algorithm

- **Compact encoding.** The index register needs $\lceil \log_2 N \rceil$ qubits. A one-hot permutation encoding needs n^2 per convoy: 100 qubits for 10 sites on one convoy, against 22 here (Section 7.5).
- **Natural fit to the objective.** The makespan is a maximum over convoys, not a quadratic form, so a QUBO needs auxiliary variables and tuned penalties. Grover search accepts an arbitrary classical cost.
- **Provable query reduction.** Dürr–Høyer needs $O(\sqrt{N})$ oracle queries against N for exhaustive search. QAOA, used in our earlier evacuation study where the objective is quadratic (Appendix A), has no such guarantee.

5.4 Warm start

A cheap classical heuristic (cheapest insertion, then relocate and 2-opt) proposes a plan, which becomes the initial threshold, as in Grover adaptive search (Gilliam et al., 2021). Its evaluations are counted in the coverage figure and reported separately, so the comparison is not flattened.

5.5 Simulation and verification

Up to 12 qubits the search runs as real Qiskit circuits on the statevector simulator. At 21–22 qubits and thousands of iterations we use the exact closed-form Grover statistics: after j iterations the probability of measuring a marked plan is $\sin^2((2j+1)\theta)$ with $\sin\theta = \sqrt{(M/2\text{^}q)}$, uniform within the marked and unmarked sets. This is the circuit's exact output distribution, and a unit test checks it against Qiskit statevector simulation of the real circuit.

5.6 Quantum assumptions and limitations

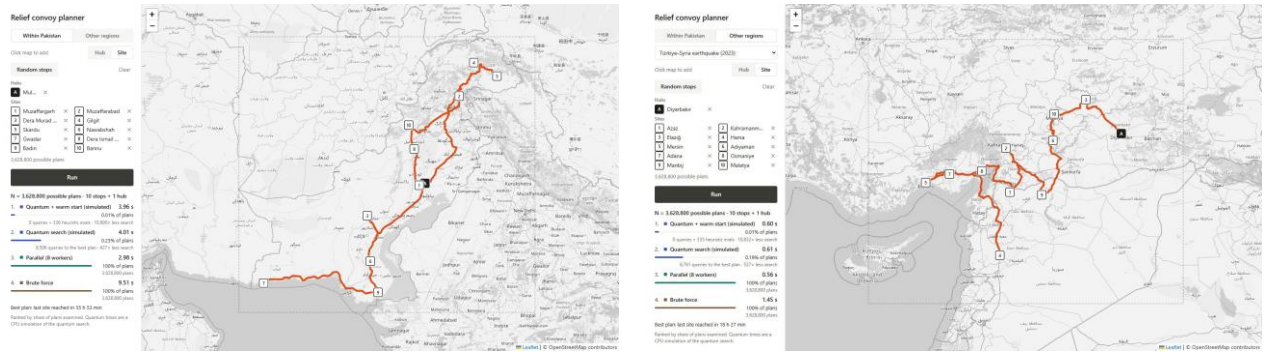
- The oracle is emulated: all N plan costs are computed classically and queries are counted. A real device needs a reversible circuit that computes the cost of a plan from its index (or QRAM with a comparator); its ancilla qubits and depth are not estimated here.
- Error-correction overheads may cancel a quadratic speedup on early fault-tolerant hardware (Babbush et al., 2021).
- The diagonal oracle stand-in used for the circuit measurements in Section 7.5 costs about 2^q CNOTs per iteration, which no real oracle would.

6. Implementation and reproducibility

Module	Role
roadgraph, geo, pool, regions_data	OSRM travel-time matrices and road geometry (cached), 49 Pakistani cities and 3 cross-border regions, random scenarios
vrp, run	Plan indexing, vectorised evaluation, brute force, parallel brute force, Dürr–Høyer search, warm start
dp_baseline	Exact subset dynamic-programming baseline (Held–Karp per convoy plus min–max combine)
server	FastAPI: /api/regions, /api/random, /api/run, /api/cases
bench_ws, tsplib_bench, qubit_study, region_runs	Benchmarks that produce backend/results/*.json
raptor, model, qaoa, cluster	Earlier evacuation study (RAPTOR transit routing and QAOA), Appendix A
frontend/ (React, Leaflet)	Map UI: random stops, ranked results, animated routes

Table 2. Code organisation (backend/src/qroute and frontend/src).

Setup: install uv and Node 18+, then run `uv sync` and `uv run qroute-server` in backend/, and `npm install` and `npm run dev` in frontend/. All benchmarks use fixed seeds and cached OSRM responses, so the numbers in this report can be regenerated offline; the unit tests (pytest) check the plan encoding, the Grover statistics against Qiskit, the DP baseline against enumeration, the TSPLIB distance code and the region generators.



Pakistan: 1 hub, 10 sites

Türkiye–Syria: 1 hub, 10 sites

Figure 2. The web app after a run. Results are ranked by share of plans examined; the best routes are drawn on real roads.

7. Results

7.1 Four regions, six countries

Region	Brute force (s)	Parallel (s)	Subset DP (s)	Quantum, simulated (s)	Quantum + warm start, simulated (s)	All find the optimum
Pakistan floods	0.64	0.31	0.020	1.42	0.77	yes
Türkiye–Syria earthquake	3.15	1.91	0.035	1.21	1.23	yes
Bengal floods (Bangladesh + India)	0.18	0.10	0.041	0.60	0.58	yes
Nepal–India Himalaya earthquake	0.17	0.09	0.037	0.70	0.75	yes

Table 3. Wall-clock time per method on the development laptop. The quantum times are the cost of simulating the search on a CPU, not hardware time.

Region	Brute force and parallel: plans	Subset DP: steps	Quantum: queries to best plan	Quantum coverage	Less than exhaustive	Warm start: queries + heuristic evals	Warm coverage
Pakistan floods	3,628,800	102,627	8,326	0.229%	436×	5,096 + 354	0.150%
Türkiye–Syria earthquake	3,628,800	102,400	6,761	0.186%	537×	0 + 335	0.009%
Bengal floods (Bangladesh + India)	1,814,400	62,274	3,681	0.203%	493×	3,499 + 141	0.201%
Nepal–India Himalaya earthquake	1,814,400	62,274	4,043	0.223%	449×	3,065 + 303	0.186%

Table 4. Work done by each method. Coverage is the share of all N plans examined before the best plan was held.

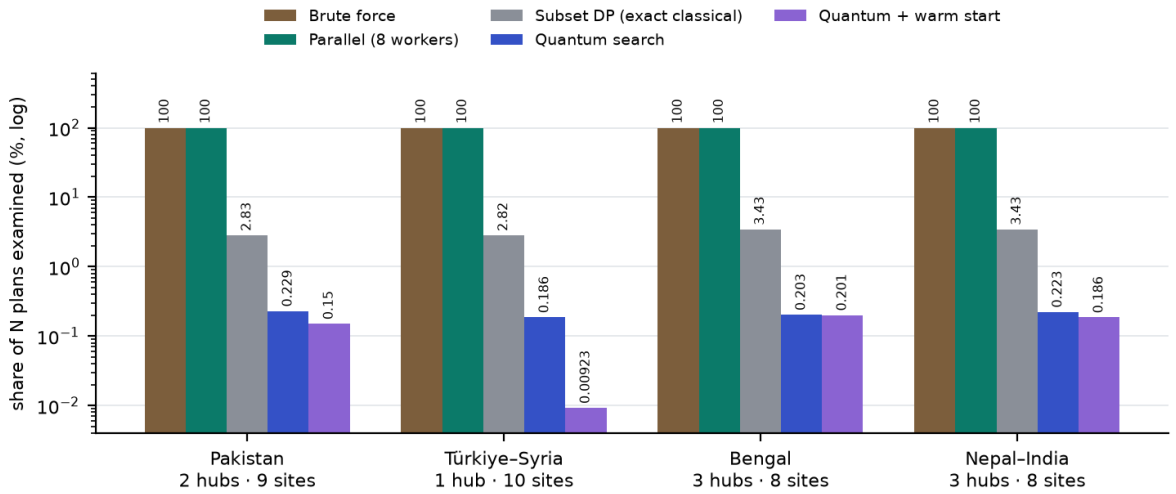


Figure 3. Share of plans examined in the four region runs (log scale). Exact subset DP is shown for reference: it examines about 3% of the plans, ten times more than the quantum queries but far cheaper per step.

7.2 Scaling with problem size

Convoys k	Sites n	Plans N	Qubits	Simulation	Median queries to optimum	N ÷ queries	Success
1	4	24	5	exact statevector	14.0	1.7×	100%
1	5	120	7	exact statevector	31.5	3.8×	100%
1	6	720	10	exact statevector	113.5	6.3×	100%
1	7	5,040	13	closed form	253.5	19.9×	100%
1	8	40,320	16	closed form	520.5	77.5×	100%
1	9	362,880	19	closed form	2,242.0	161.9×	100%
1	10	3,628,800	22	closed form	5,825.5	622.9×	100%
2	4	120	7	exact statevector	40.5	3.0×	100%
2	5	720	10	exact statevector	113.5	6.3×	100%
2	6	5,040	13	closed form	274.0	18.4×	100%
2	7	40,320	16	closed form	454.0	88.8×	100%
2	8	362,880	19	closed form	2,113.5	171.7×	100%
2	9	3,628,800	22	closed form	4,676.5	776.0×	100%

Table 5. Problem-size sweep on real distances (20 seeds each). The advantage over exhaustive search grows from 1.7× at N = 24 to over 600× at N = 3.6 million; at small N there is none.

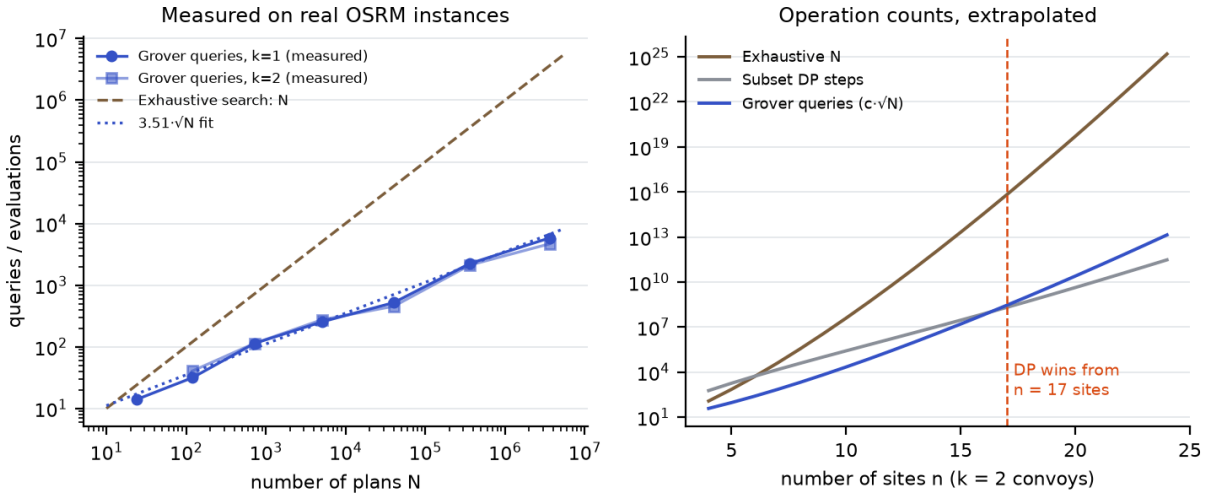


Figure 4. Left: measured Grover queries against exhaustive search, with the fit 3.51·√N. Right: extrapolated operation counts for two convoys; exact subset DP needs fewer steps than Grover from n = 17 sites.

7.3 Warm start: what it does and does not do

Region	Runs	Standard: median queries	Warm: mean queries	Standard coverage	Warm coverage (incl. heuristic)	Coverage gain	Query gain when heuristic misses	Heuristic already optimal
Pakistan floods	100	4,840	1,567	0.159%	0.0099%	12.9×	1.18×	65%
Türkiye–Syria earthquake	25	4,196	556	0.150%	0.0094%	12.3×	1.84×	80%
Bengal floods (Bangladesh + India)	25	4,616	626	0.157%	0.0094%	15.5×	1.30×	80%
Nepal–India Himalaya earthquake	25	4,840	0	0.139%	0.0078%	18.5×	n/a (all zero)	100%

Table 6. Standard against warm-started Dürr–Høyer, paired per scenario and quantum seed (20 Pakistan scenarios × 5 seeds; 5 × 5 for each other region). Every run found the optimum.

The warm start lowers total coverage by roughly 12–18×, but almost all of that gain comes from the classical heuristic hitting the optimum directly in 65–100% of scenarios. When the heuristic misses, the warm-started search needs only about 1.2–1.8× fewer queries, and in some paired runs more. We therefore present the warm start as a practical hybrid, not as evidence for a deeper quantum advantage.

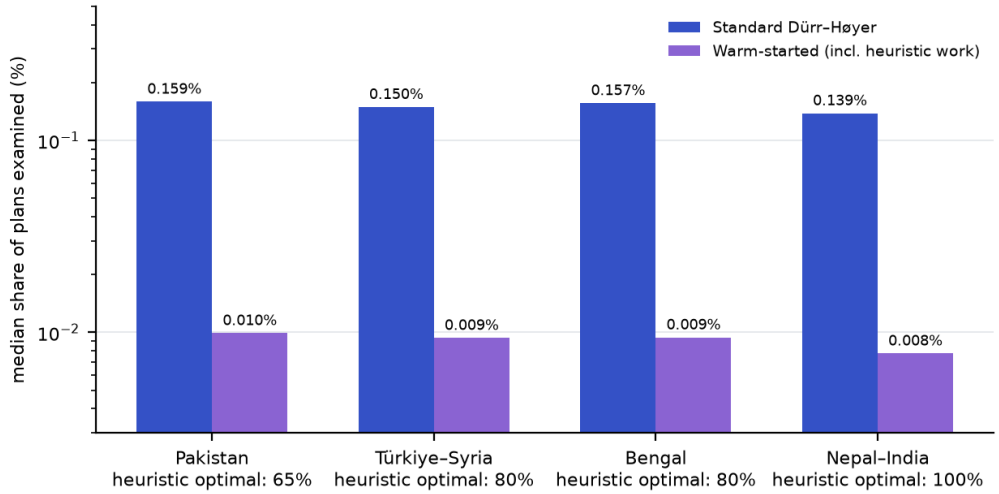


Figure 5. Median share of plans examined, standard against warm-started search.

7.4 Validation on a published benchmark (TSPLIB burma14)

To check our cost and distance code against an external reference, we implemented the TSPLIB GEO distance and ran exact Held–Karp dynamic programming on burma14 and ulysses16, reproducing the published optima 3,323 and 6,859. We then ran the quantum search on the first n cities of burma14 (closed tours, (n−1)! orders), 20 seeds per row.

Cities n	Tours N	Qubits (one-hot)	Nearest-neighbour gap	Cold: median queries	N ÷ queries	Warm: median queries	Success
5	24	5 (25)	0.0%	9.0	2.7×	0.0	100%
6	120	7 (36)	6.8%	17.0	7.1×	15.5	100%
7	720	10 (49)	16.8%	53.5	13.5×	54.5	100%
8	5,040	13 (64)	21.4%	177.5	28.4×	227.0	100%
9	40,320	16 (81)	20.3%	538.0	74.9×	546.5	100%
10	362,880	19 (100)	15.7%	1,406.5	258.0×	1,204.0	100%
11	3,628,800	22 (121)	15.6%	4,534.0	800.4×	4,883.5	100%

Table 7. burma14 prefixes. Qiskit statevector circuits for n ≤ 7 (q ≤ 10), exact closed-form statistics above. Warm start helps only at n = 5; elsewhere it is within noise.

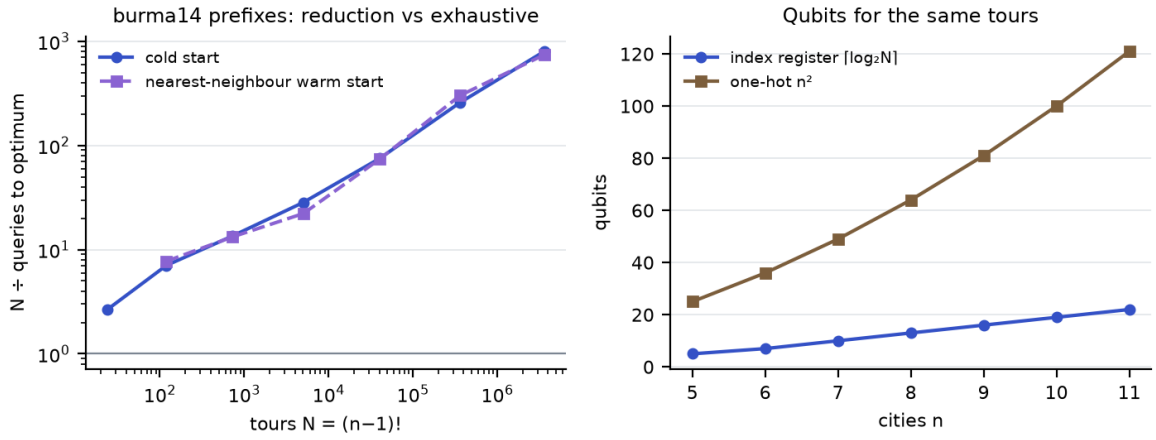


Figure 6. Left: reduction against exhaustive enumeration on burma14 prefixes. Right: qubits needed by the index register and by a one-hot encoding for the same tours.

Published Grover-style TSP studies report 31 qubits for a 7-node TSP with 86.7% optimal-solution success (arXiv 2212.02735) and 24 and 56 qubits for 4 and 5 cities with a standard Grover encoding (ICTC 2025). The index register needs 10 qubits for 7 cities. These encodings are different trade-offs: ours pushes the work into a reversible cost oracle that we do not build.

7.5 Qubits: how many we used and what changes with the count

We use q = ⌈$\log_2 N$⌉ qubits: 21 for 1.8 million plans and 22 for 3.6 million. To test whether this is the right number we ran three experiments (backend/results/qubit_study.json).

Instance	+0 qubits (minimal)	+1	+2	+3	+4
k=1, n=7 (N=5,040)	235 (1.00×)	346 (1.47×)	614 (2.61×)	709 (3.02×)	867 (3.69×)
k=1, n=10 (N=3,628,800)	6,304 (1.00×)	6,550 (1.04×)	9,572 (1.52×)	14,764 (2.34×)	21,848 (3.47×)
k=2, n=7 (N=40,320)	698 (1.00×)	946 (1.35×)	1,432 (2.05×)	1,698 (2.43×)	2,906 (4.16×)

Table 8. Experiment A, register padding. Median queries to the optimum when the register is larger than ⌈$\log_2 N$⌉ (30 seeds). Success stayed at 100% throughout.

Experiment A answers the question about the optimal number of qubits. Fewer than ⌈$\log_2 N$⌉ qubits cannot index all plans, and every extra qubit only adds unmarked states, so the minimal register had the lowest median queries in all three instances. Each extra qubit multiplies the queries by about 1.4 (+4 qubits gave 3.5–4.2×) without changing the success rate.

Qubits q	Depth per iteration	CNOTs per iteration	of which oracle stand-in	of which diffuser	Statevector bytes
3	29	12	6	6	128
4	55	28	14	14	256
5	125	66	30	36	512
6	231	144	62	82	1,024
7	339	248	126	122	2,048
8	667	424	254	170	4,096
9	1205	736	510	226	8,192
10	1979	1312	1022	290	16,384
11	4329	2408	2046	362	32,768
12	8451	4536	4094	442	65,536

Table 9. Experiment C, circuit resources of one Grover iteration in Qiskit (transpiled to cx, rz, sx, x). CNOTs grow about 1.9× per qubit, dominated by the generic diagonal oracle stand-in; the diffuser grows far more slowly. Memory doubles per qubit, which is why we switch to closed-form statistics above 12 qubits.

Convoys k	Sites n	Plans N	Index register (ours)	One-hot n^2	One-hot $(n-1)^2$
1	4	24	5	16	9
1	5	120	7	25	16
1	7	5,040	13	49	36
1	10	3,628,800	22	100	81
2	7	40,320	16	49	36
2	9	3,628,800	22	81	64

Table 10. Qubit counts of alternative encodings of the same problems. The index register is far smaller but excludes the ancillas of a reversible cost oracle.

Conclusion on qubits: the optimal register is the minimal ⌈$\log_2 N$⌉; a more compact register does not exist, and larger registers cost queries. The practical limit on today's classical simulators is the statevector memory of 16·2^q bytes, which is the reason for the closed-form statistics at 21–22 qubits.

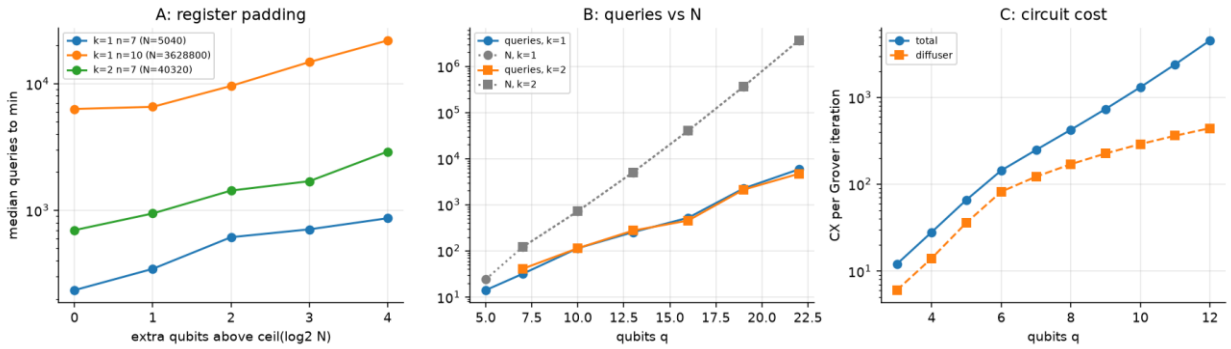


Figure 7. Qubit study. Left: queries against padding qubits. Middle: queries and N against qubits. Right: CNOTs per Grover iteration.

8. When quantum search helps, and when it does not

Against exhaustive search the picture is clear and grows with problem size. Against the best classical exact algorithm it is not, and the report would be incomplete without that comparison. Exact subset dynamic programming solves the min–makespan problem in about $k \cdot 2^n \cdot n^2 + (k-1) \cdot 3^n$ steps; using the measured Grover constant $c = 3.51$ (queries $\approx c \cdot \sqrt{N}$), Table 11 compares step counts for two convoys.

Sites n (k = 2)	Exhaustive: N	Subset DP: steps	Grover: c·√N queries
8	362,880	39,329	2,114
10	4.0×10^7	263,849	22,167
12	6.2×10^9	1.7×10^6	276,860
14	1.3×10^12	1.1×10^7	4.0×10^6
16	3.6×10^14	7.7×10^7	6.6×10^7
18	1.2×10^17	5.6×10^8	1.2×10^9
20	5.1×10^19	4.3×10^9	2.5×10^10

Table 11. Operation counts, extrapolated. Exact DP needs fewer steps than plain Grover from n = 17 sites (k = 1), 17 (k = 2) and 16 (k = 3). One Grover query would additionally need a reversible plan-cost evaluation, while a DP step is a few arithmetic operations.

- **Where it helps.** Unstructured or black-box plan spaces where only the cost of a complete plan can be evaluated (for example a simulator-based cost with damage, time windows and capacities). There the √N query reduction over exhaustive search is real: 350–800× at 1.8–3.6 million plans, and it grows with N.
- **Where it does not.** When the cost decomposes (as here, with free-flow travel times) classical DP is exact, instant at these sizes (20–41 ms), and needs fewer steps than plain Grover above about 16–17 sites.
- **Route to a real advantage.** Hybrid algorithms that combine Grover search with a partial dynamic-programming table reach $O^*(1.728^n)$ for TSP, below the $O^*(2^n)$ of Held–Karp (Ambainis et al., 2019). Applying them to the multi-convoy problem is future work.

9. Limitations

- **Simulation.** All quantum results are classical simulation. Above 12 qubits we use exact closed-form Grover statistics (verified against Qiskit at small sizes) because a 22-qubit circuit with thousands of iterations is impractical to simulate gate by gate.
- **Emulated oracle.** Plan costs are computed classically; no reversible cost circuit, ancilla count or fault-tolerant resource estimate is provided.
- **Baselines.** Brute force is a weak baseline. Exact subset DP is faster in wall-clock time and, beyond about 16–17 sites, in step count. The query reduction is real but is measured against exhaustive search.

- **Warm start.** Its coverage gain is mostly the classical heuristic's own accuracy (Section 7.3).
- **Model realism.** Travel times are free-flow OSRM estimates between city centres. Flooded or damaged roads, vehicle capacity, relief quantities and time windows are not modelled.
- **Statistics.** Region runs use one seeded scenario each; the qubit study uses fixed city subsets and 20–30 seeds per cell; the warm-start benchmark uses 20 Pakistan and 5 scenarios per other region.
- **Scale of the demonstrator.** 1–3 hubs, at most 10 sites and N ≤ 4 million, limited by the memory of the classical oracle table.
- **External service.** The public OSRM server is rate-limited; responses are cached in the repository so results reproduce offline.

Appendix A. Earlier study: evacuation assignment with RAPTOR and QAOA

Our first approach routed evacuee groups over a synthetic bus network laid on Karachi's real coordinates. RAPTOR (classical, exact) finds Pareto-optimal transit journeys for each group; a QUBO then picks one journey per group to minimise person-minutes plus bus crowding and shelter load, with exactly one journey per group. Because this objective is quadratic, QAOA is a natural fit. We compared textbook QAOA with a one-hot penalty against a constraint-preserving XY-mixer QAOA, both trained with a CVaR objective and sampled with 4096 shots on Qiskit Aer.

Variant	Feasible samples	Optimal samples	Approximation ratio	Optimum found
Penalty, p=1	39%	0.6%	0.52	yes
Penalty, p=2	58%	0.7%	0.52	yes
Penalty, p=3	98%	1.3%	0.52	yes
XY mixer, p=1	100%	15.1%	0.79	yes
XY mixer, p=2	100%	6.7%	0.86	yes
XY mixer, p=3	100%	16.8%	0.83	yes

Table 12. 12-qubit instance (4 groups × 3 journeys, 81 feasible assignments). A random feasible guess samples the optimum 1.2% of the time.

The XY mixer keeps every sample feasible by construction. On a 36-variable instance, clustered XY-QAOA (≤12-qubit sub-problems solved in turn) reached the exact optimum (gap 0.00%), while greedy was 1.61% above it, worse than no coordination (1.03%). What did not work well: penalty QAOA needed depth to become feasible and tuned penalties, and XY p=2 sampled the optimum less often than p=1, a sign that the classical optimiser (COBYLA with three restarts) got stuck in local optima.

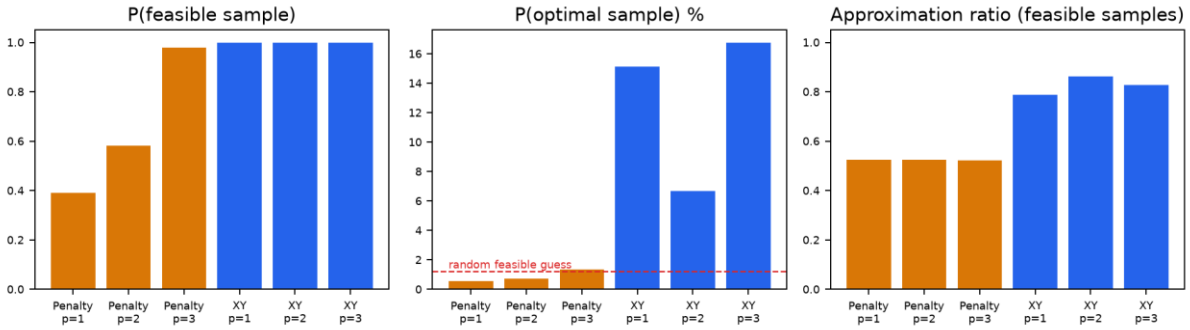


Figure 8. QAOA study: feasibility, optimal-sample probability and approximation ratio, penalty against XY mixer.

References and credits

1. C. Dürr, P. Høyer. A Quantum Algorithm for Finding the Minimum. arXiv:quant-ph/9607014, 1996.
2. M. Boyer, G. Brassard, P. Høyer, A. Tapp. Tight bounds on quantum searching. Fortschr. Phys. 46, 1998.

3. L. K. Grover. A fast quantum mechanical algorithm for database search. STOC 1996.
4. A. Gilliam, S. Woerner, C. Gonciulea. Grover Adaptive Search for Constrained Polynomial Binary Optimization. Quantum 5, 428, 2021.
5. R. Babbush et al. Focus beyond quadratic speedups for error-corrected quantum advantage. PRX Quantum 2, 010103, 2021.
6. A. Ambainis et al. Quantum speedups for exponential-time dynamic programming algorithms. SODA 2019 (arXiv:1807.05209).
7. arXiv:2212.02735 (Grover adaptive search for TSP); ICTC 2025 F01-4 (Grover TSP qubit counts); arXiv:2106.05948 and 2012.04984 (D-Wave studies on TSPLIB).
8. M. Held, R. Karp. A dynamic programming approach to sequencing problems. 1962. G. Reinelt. TSPLIB, 1991.
9. E. Farhi et al., 2014 (QAOA); S. Hadfield et al., 2019 (XY mixer); P. Barkoutsos et al., 2020 (CVaR); D. Delling, T. Pajor, R. Werneck, 2015 (RAPTOR).
10. Qiskit and Qiskit Aer (IBM); FastAPI; React; Leaflet; NumPy. Road data, routing and map tiles © OpenStreetMap contributors (ODbL), routing by OSRM. Logos from simple-icons (CC0). All code was written for this project; no tutorial code was copied.